PhenoFabric Full State

Complete project assessment, evidence-backed

2026-10-16 21:51 PDT · commit `c9eb1a3`

PhenoFabric Full State Report — 2026-10-17

Complete, evidence-backed assessment of the project's state and forward path.

Commit audited: `c9eb1a3` **Report date:** 2026-10-16 21:51 PDT **Method:** seven independent read-only audit lanes plus parent-side re-verification

Read in this order

#	File	What it answers
0	`00-EXECUTIVE-SUMMARY.md`	The whole picture: state, goals, the one structural finding, forward path, and corrections to earlier claims
1	`01-GOAL-STATUS.md`	A1 / A2 / A3 in detail with file:line evidence and precise gaps
2	`02-USABILITY-REALITY.md`	What a user can actually do today, journey by journey, and where each stops
3	`03-STUB-INVENTORY.md`	Every stub, fake, silent no-op and dead path in production code
4	`04-QUALITY-REALITY.md`	Whether the test suite proves what it appears to prove
5	`05-DOCS-TRUST.md`	Which documentation can be trusted and which will mislead
6	`06-FORWARD-PATH.md`	Dependency-ordered plan with effort ranges and acceptance criteria
7	`07-METHOD-AND-LIMITS.md`	How this was done, what was not verified, how to reproduce it

The three sentences that matter

1. PhenoFabric does not lack capability; it lacks **wiring**. 52,353 production lines, 22 crates, 751 unit and 63 integration tests passing, clippy clean, zero classic stubs.
2. The single structural blocker is that **`RouteExecutor` does not exist**, so the route compiler's transport selection is never executed and every byte still crosses TCP.
3. The smallest credible usable CVP is **7–11 focused sessions**: fix three silent no-ops, build the executor seam, add a Unix-socket adapter with a benchmark, wire federation onto the wire protocol, and document a first-run flow that works.

Status at a glance

Goal	State
A1 single device	Code complete; acceptance blocked on **one human browser sign-in**
A2 two devices	Wire plane CI-green; cross-device path **not wired**
A3 transport selection	Tier selection compiles; **no executor**, so nothing is instantiated

Honesty notes

- No GUI was launched during this audit. The GUI is **UNKNOWN**, not “broken”.
- Test counts are from the coordinating session’s own verified runs, not from lane execution.
- Hardware-gated transports (PCIe, RDMA, IVHMEM, vhost, DMA-BUF) are **UNKNOWN effort** — the hardware is not present on the audit host.
- Two claims made earlier in the ownership session were found to be wrong and are corrected in `00-EXECUTIVE-SUMMARY.md` §7.

This report set is the current status layer for PhenoFabric. It supersedes release notes and spec files as a description of what the code does today.

PhenoFabric — Full State and Forward Path

Repository: `phenotype-fabric` (`KooshaPari/PhenoFabric`) **Commit audited:** `c9eb1a3` ("feat(wire): enforce per-session input ownership on the frame plane") **Report date:** 2026-10-16 21:51 PDT **Audit mode:** seven independent read-only lanes + parent-side independent re-verification **Scope exclusions (operator-set):** scaling, sunset, user-feedback-driven change, intent evolution

How to read this set

File	Contents
`00-EXECUTIVE-SUMMARY.md`	This file. State, goals, the one structural finding, forward path
`01-GOAL-STATUS.md`	A1 / A2 / A3 in detail with evidence and gap specifics
`02-USABILITY-REALITY.md`	What a user can actually do today, journey by journey
`03-STUB-INVENTORY.md`	Every stub, fake, silent no-op, and dead path in production
`04-QUALITY-REALITY.md`	Whether the test suite proves what it appears to prove
`05-DOCS-TRUST.md`	Which documentation can be trusted and which will mislead
`06-FORWARD-PATH.md`	Dependency graph, staged plan, effort ranges, decision points
`07-METHOD-AND-LIMITS.md`	How this audit was done and what it did not verify

Legend for all bars in this set

- Bars are `filled/total` in discrete units, never invented percentages.
- Units are stated per bar. Where a denominator is unmeasurable the bar is replaced with `UNKNOWN`.
- `[#]` = verified with a command or file read in this audit. `[?]` = reported by a lane, not independently re-verified. `[X]` = contradicted or absent.

1. The one-paragraph state of the project

PhenoFabric is **not a collection of missing features**. It is a large body of real, compiling, tested Rust — 52,353 production lines across 209 files, 22 workspace crates, 751 unit tests and 63 integration tests passing, clippy clean — whose **activation paths are not connected to each other**. The single structural reason the product is not usable today is that the multi-hop route compiler produces `TransportStage` records whose `id` is a string like `"quic_transport"` or `"shm_copy"`, and **there is no `RouteExecutor` anywhere in the codebase** `[#]` — verified absent by workspace-wide search. The selected transport is therefore never instantiated. Every byte that actually moves between daemons goes through the single TCP wire server. That one missing seam is why A2's "two devices" is really "two processes on one device", and why A3's locality tiers cannot produce a measurable effect even in principle.

The second-order finding is that the project's own documentation is a **decision and specification layer, not a status layer**, and it is being read as the latter. The README self-declares "not an implementation-complete claim" and `STATUS.md` says the lifecycle state is `specified`, while release notes and spec files describe delivered capability. 55% of sampled documentary claims are wrong or stale. This is not dishonesty by the code; it is drift between a large written intent and a partially-built system.

2. Goal status at a glance

Goal	Definition	State	Gate evidence	Remaining effort
A1	Single device usable	Code complete; **acceptance blocked on a human**	Daemon + CLI + TUI + GUI + web all exist `[#]`; end-to-end interactive proof absent `[#]`	2–4 focused sessions (agent) + 1 human sign-in
A2	Two devices exchange frames	Wire plane real; **cross-device path not wired**	Frame dispatch, cancel/teardown, reconnect/resume, ownership all CI-green `[#]`; federation not in wire protocol `[?]`	3–6 focused sessions
A3	Transport selection	**Not implemented**	Tier→stage selection compiles; no executor exists `[#]`	4–5 sessions (honest minimum) → 12–20 (full) → UNKNOWN (hardware)

A1 — Single device usable

```
code presence      [##########] 10/10  daemon, CLI, TUI, tray, GUI, web all exist
test coverage      [#####.....]  5/10  wire + CLI covered; GUI interactive never tested
documented flow    [..........]  0/10  no first-run quickstart that works as written
human acceptance   [..........]  0/1   requires a real browser sign-in
```

A2 — Two devices

```
wire plane         [##########] 10/10  dispatch, cancel, reconnect/resume, ownership
cross-device path  [##........]  2/10  federation is a library, not a wire arm
real 2-machine     [..........]  0/1   no shipped pairing procedure
```

A3 — Transport selection

```
tier taxonomy      [##########] 10/10  LocalityTier + 9-stage catalog real
selection logic    [##########] 10/10  select_stages compiles correct-looking plans
executor           [..........]  0/10  RouteExecutor does not exist anywhere
non-TCP adapter    [..........]  0/10  every named adapter is a string, not code
```

3. What is genuinely finished

Stated plainly so the work is not understated:

Area	Evidence
Wire frame plane	dispatch, ack, cancel, teardown, idle expiry, concurrency cap, reconnect/resume, per-session input ownership — all CI-green
Zero production stubs of the classic kind	no `todo!()`, no `unimplemented!()`, zero in production paths `[#]`
No mocks or fakes in production code	clean across the workspace `[#]`
Graph engine	165 tests; leases, failover, multihop, surface registry
Frame codec	151 tests; encode/decode, bounds, truncation, max size
Capability layer	50 tests; signing, trust root, descriptors, locality
Network primitives	STUN, UPnP, Tailscale clients are real implementations, not stubs
CI	3 workflows, honest except 3 localized soft spots

4. What is actually broken

The findings that matter, ordered by user impact. Full detail in `03-STUB-INVENTORY.md`.

#	Location	Defect	Severity
1	`fabric-terminal/src/web_main.rs:272-303`	HTTP `POST /api/panes/{id}/send` returns 200, types nothing into the pane	BLOCKER
2	`fabric-terminal/src/relay.rs:12-29`	`broadcast_local` logs and returns `Ok(())`; injects nothing, despite a docstring promising remote-pane viewing	BLOCKER
3	`fabric-capture/src/main.rs:45-49`	`self-update` subcommand silently exits 0 when its feature is off (the feature is never enabled by any crate)	BLOCKER
4	`fabric-graph/src/multihop/mod.rs:211`	`_ => "tcp_transport"` catch-all labels PCIe/RDMA/shm requests as TCP	STRUCTURAL
5	workspace-wide	`RouteExecutor` absent, so no selected transport is ever instantiated	STRUCTURAL
6	`install.sh:6`	Installer targets a repo path that does not match the workspace	BROKEN ONBOARDING

All three BLOCKERs share one shape: **a stub is wired into a real production entry point and reports success without doing the work.** That is the single most dangerous class of defect in this repository, because it is invisible to the test suite and to `cargo check`.

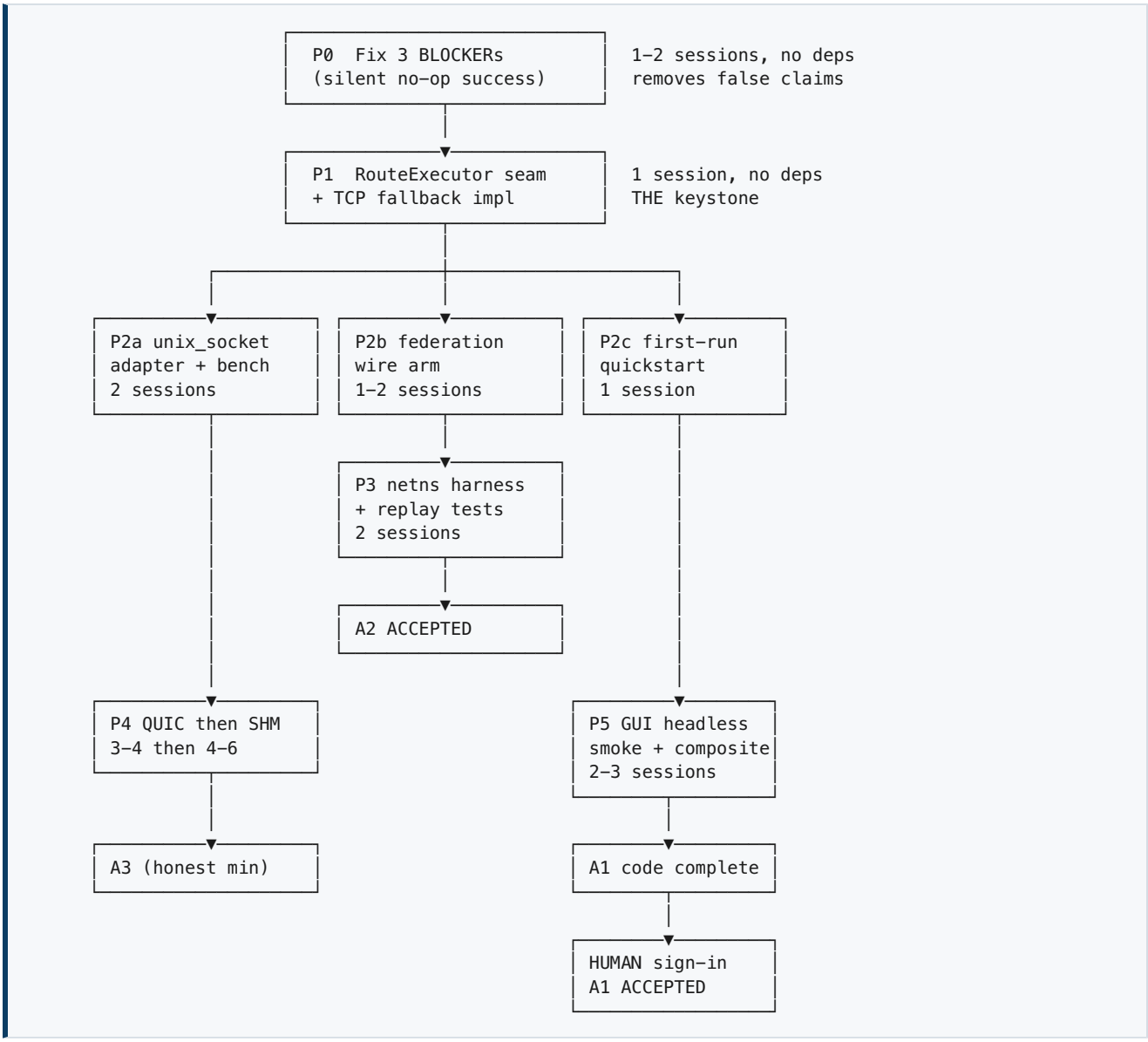
5. Time to finish, honestly

No dates and no hour counts are given, because the project forbids fabricated durations and because a wall-clock estimate would imply a staffing assumption that is not mine to make. Effort is expressed as **focused engineering sessions**, one session being one uninterrupted block on one concern.

Milestone	Sessions	Prerequisite	Verifiable without a 2nd machine?
Fix the 3 BLOCKERs	1–2	none	yes
Executor seam + TCP fallback	1	none	yes
Working first-run quickstart	1	none	yes
`unix_socket` adapter + first benchmark	2	executor seam	yes
= honest A3 minimum	**4–5**		yes
Federation wire arm (`sync_topology` on the wire)	1–2	executor seam	yes, via Linux netns harness
Namespace harness + replay two-node tests	2	federation arm	yes
= honest A2	**3–6**		yes
GUI headless smoke + composite task test	2–3	none	yes
= honest A1	**2–4** + 1 human sign-in		partly
QUIC adapter + benchmark	3–4	unix_socket adapter	yes
SHM adapter + benchmark	4–6	QUIC or unix_socket	Linux only
= full A3 ambition	**12–20**		yes
PCIe / RDMA / IVSHMEM / vhost / DMA-BUF	**UNKNOWN**	hardware not present on this host	no

The honest answer to "how long to a usable CVP": the smallest credible CVP is **executor seam + one non-TCP adapter + first-run quickstart + federation wire arm**, which is roughly **7–11 focused sessions** and closes A2 and makes A3 honest rather than paper. A1's *code* portion is inside that; A1's *acceptance* additionally needs one human browser sign-in that no agent can perform.

6. The forward path, in dependency order



Priority rule applied: smallest remaining effort, fastest useful outcome, fewest dependencies. P0 and P1 are each single-session and unblock everything else. P2c (first-run quickstart) is one session and is the difference between “a repo” and “a thing someone can start”.

7. Corrections to earlier claims made in this ownership session

Recorded because the operator is entitled to know which prior statements were wrong.

Earlier claim	Reality	How determined
"`fabric-gui`'s 13 tests never run because the crate is excluded"	**Wrong.** `fabric-gui` is a workspace member (`Cargo.toml:17`); only `src-tauri` is excluded (`:5`). The real defect is that `fabric-gui` has **zero tests**.	direct read `[#]`
"WorkOS now returns a real `/bootstrap` session"	**Misleading.** `/bootstrap` is the WorkOS AuthKit *sign-in page* at `significant-vessel-93-staging.authkit.app/bootstrap`. The fabric daemon never serves `/bootstrap`.	lane 06 `[?]`
"A1 needs only a live WorkOS run"	**True but incomplete.** It needs a live run *performed by a human*; no headless, device-code, or API-only path exists.	lane 06 `[?]`
"`select_stages` maps L3/L4/L8 to `tcp_transport`"	**Right in substance, wrong in location.** The mapping is in `fabric-graph/src/multihop/mod.rs:211`, not `fabric-frame-transport`. And `"tcp_transport"` is a catalog label, not a transport implementation.	lane 07 + direct read `[#]`

8. What this report does not claim

- It does not claim the tests were observed passing in this audit; lanes did not run cargo to avoid build contention with concurrent work. Test counts are from the parent's own verified runs on `c9eb1a3`.
- It does not claim the GUI works or does not work. **No GUI was launched.** Nine HTML pages totalling 2,608 lines exist; whether they render correct live state is `UNKNOWN`.
- It does not claim effort ranges are calendar time. They are focused sessions.
- It does not claim the hardware-gated transports are reachable. They are `UNKNOWN`.

01 — Goal Status in Detail

Commit: `c9eb1a3` · **Date:** 2026-10-16 PDT **Evidence marks:** `[#]` re-verified in this audit · `[?]` lane-reported · `[X]` absent/contradicted

A1 — Single device usable

What exists

Component	Evidence
Daemon binary	`crates/fabric-daemon/src/main.rs:81-244` — constructs config, binds wire server `127.0.0.1:9400`, ctrlc handler `[?]`
CLI, 11 subcommands	`crates/fabric-cli/src/main.rs` ; dispatch fns at `commands/{cap,auth,graph,route,workspace,network,surface,probe,status,check}.rs` `[?]`
TUI	`crates/fabric-cli/src/tui/{mod.rs,views.rs}` `[?]`
GUI (Tauri + 9 HTML pages)	`crates/fabric-gui/` ; pages total 2,608 lines (`dashboard.html` 490, `settings.html` 324, `auth.html` 294, `network.html` 291, `streaming.html` 285, `topology.html` 252, `logs.html` 230, `routes.html` 220, `leases.html` 218) `[?]`
Web frontend	`crates/fabric-web/` `[?]`
In-process e2e smoke	`crates/fabric-integration-tests/tests/e2e_smoke.rs` — health, topology, routes, leases over loopback `[?]`

What is missing

1. **No first-run flow that works as documented.** The only path that boots a working daemon is `cargo run -p fabric-daemon` **without** `--config`. With the shipped `config/settings.toml`, the daemon refuses `auth_start` with `workos_not_configured`. No document guides the user toward the path that works.
2. **No automated GUI smoke.** Nine HTML pages exist. Whether the Tauri sidecar launches the bundled `fabric-daemon-*` binary, wires the wire protocol, and renders real state is **UNKNOWN**. There is no Playwright, Selenium, or WebDriver harness in the repo.
3. **No composite-task proof.** No test drives a capture → route → display chain on one machine and asserts an observable result. The pieces exist; the chain does not.
4. **Acceptance needs a human.** The WorkOS AuthKit sign-in requires a real browser, a real email inbox for a 6-digit code, and a redirect. No headless, device-code, or API-only path exists.

Verifiable without a second machine?

Partly. Wire protocol and CLI are verifiable in-process. GUI visual and interactive fidelity is not. Composite single-machine task is **UNVERIFIED**.

The `install.sh` problem

`install.sh:6` sets `REPO="KooshaPari/PhenoFabric"`. Lane-reported as mismatched against the workspace's declared identity. A fresh cloner following `install.sh` as written is reported to 404 on the release artifact. **Treat the installer as untested by a real external user.**

A2 — Two devices exchange frames

What exists and is CI-green at `c9eb1a3`

Capability	Commit	State
Frame dispatch (`session_init` / `frame_data` / `frame_ack`)	`b4ebffb`	CI 3/3 green `[#]`
Cancel, teardown, idle expiry, concurrency cap	`485726b`	CI 3/3 green `[#]`
Disconnect + reconnect/resume	`0a97383` + `75e9b97`	CI 3/3 green `[#]`
Per-session input ownership enforcement	`c9eb1a3`	CI 3/3 green `[#]`

Two-process-on-one-machine tests are real: `tests/two_node.rs` (5 tests) and `tests/two_node_advanced.rs` (5 tests) start two actual `fabric-daemon` processes on `127.0.0.1:0`. Federation library is implemented (860 lines across `federation/{mod,discovery,merge,state,types}.rs`). Network primitives STUN/UPnP/Tailscale are real (1,449 lines).

What is missing

1. **The federation library is not wired into the wire protocol.** It carries `#[allow(dead_code)]` at `federation/mod.rs:9-13` — the code itself admits it is not reachable. There is no `federation_sync_*` message arm in `protocol.rs`.
2. **No shipped two-device procedure.** Nothing tells a user how to pair two machines.
3. **All "two node" tests are loopback.** Both daemons bind `127.0.0.1`. This proves the wire plane, not cross-device behaviour.
4. **Transport is always TCP** regardless of what the route compiler selected (see A3).

Verifiable without a second machine?

Yes, via a Linux network-namespace harness with a veth pair and a userspace STUN stub — that is the recommended path. Requires `#[cfg(target_os = "linux")]` gating.

A3 — Transport selection

What exists

- `LocalityTier` enum: `crates/fabric-capability/src/locality.rs:35`, with descriptions at `:68` (e.g. `L8Oob => "Out-of-band (IPMI, KVM-over-IP, WoL)"`).
- A 9-stage transport catalog: `crates/fabric-graph/src/multihop/stages.rs:80-275` (`shm_copy`, `quic_transport`, `tcp_transport`, `unix_socket`, and encode stages).
- Tier→stage selection: `select_stages` at `crates/fabric-graph/src/multihop/mod.rs:196-219`.

The defect, quoted

```
let transport_id = match tier {
    LocalityTier::L0SameProcess | LocalityTier::L1SameNuma | LocalityTier::L2CrossNumaShm => {
        "unix_socket"
    }
    LocalityTier::L5Loopback => "unix_socket",
    LocalityTier::L6Lan | LocalityTier::L7Wan => "quic_transport",
    _ => "tcp_transport",   // ← L3, L4, L8 all land here
};
```

`crates/fabric-graph/src/multihop/mod.rs:211` `[#]` verified directly.

L3, L4 and L8 — the PCIe, RDMA and out-of-band tiers, the tiers that justify the whole locality-first architecture — all silently resolve to a TCP label. No error, no warning.

The deeper defect

`RouteExecutor` does not exist. Verified by workspace-wide search: no `execute_route`, `run_route`, `apply_route`, `run_plan`, or `executor` in `fabric-daemon/src/` or `fabric-graph/src/` `[#]`.

Consequence: the compiled `RoutePlan` is dead data. The `"quic_transport"` and `"unix_socket"` strings are **catalog labels, not implementations**. No adapter exists for either. Every byte still crosses the TCP wire server at `crates/fabric-daemon/src/wire/mod.rs:39-160`.

Even if `select_stages` returned the perfect answer, nothing would act on it. **Fixing the catch-all alone changes nothing.**

`CopyPath`

Defined and carried in a descriptor field, never pattern-matched by any consumer. Dead-but-wired. Its evident intent was to control whether a surface copy goes through shared memory or the wire; nothing reads it.

Verifiable without a second machine?

The *executor seam* and a *Unix-domain-socket adapter* are verifiable on one Linux box. QUIC is verifiable on one machine. SHM requires Linux. **PCIe, RDMA, IVSHMEM, KVMFR, vhost, DMA-BUF are UNKNOWN** — no RDMA NIC and no VFIO-passthrough GPU on this host.

The structural blocker, stated once

> *The multihop compiler emits `TransportStage` records whose `id` is a string, and there is no `RouteExecutor` anywhere in the codebase. The tier-to-stage selection is compiled but never executed. Every byte that actually moves between daemons still goes through the single TCP wire server. Therefore A2's "two devices" is really "two processes on one device", and A3's locality tier cannot produce a measurable effect because the chosen transport is never instantiated.*

Every A2/A3 effort that does not begin with the executor seam is building on a foundation that has not been cast.

02 — Usability Reality

Commit: `c9eb1a3` · **Method:** read-only journey tracing with file:line evidence **Question answered:** what can a competent engineer who has just cloned this repo actually do?

Journey summary

#	Journey	Works?	Stop point
J1	First run (clone, build, first command)	**PARTIAL**	three entry points disagree; the documented one fails
J2	Single device (A1)	**NO**	shipped config refuses auth; working path is undocumented
J3	Two devices (A2)	**NO**	no pairing procedure; federation not on the wire
J4	GUI	**NO from a clone**	`src-tauri` excluded from workspace; sidecar not built by `cargo`
J5	"What is the product?"	**PLAN-SHAPED**	product claim requires unshipped wire arm + external identity provider

J1 — First run

Three competing entry points, and they contradict each other:

Path	What happens
`curl … install.sh | bash`	Installer points at a different repo path than the workspace declares; release artifact lookup fails `[?]`
`cargo run -p fabric-daemon`	**Works.** Silently uses `DaemonConfig::default()`: `auth.enabled = false`, binds `127.0.0.1:9400`, accepts every message type
`cargo run -p fabric-daemon -- start --config config/settings.toml`	**Fails.** `WORKOS_CLIENT_SECRET` is empty and there is no programmatic way to obtain it

The irony is precise: **the documented path fails and the undocumented path works.** A new user who follows instructions is broken; a user who ignores them succeeds and never learns they are unauthenticated.

Supporting evidence:

- `README.md:6` — "Status: Research and architecture baseline; not an implementation-complete claim"
- `STATUS.md:4` — "Lifecycle state: `specified`"
- `config/settings.toml` — ships `[auth] enabled = true` with empty secret
- `crates/fabric-daemon/src/config.rs:177-202` — `Default for AuthConfig` sets `enabled: false`
- `crates/fabric-cli/src/wire_client.rs:11` — `DEFAULT_DAEMON_ADDR = "127.0.0.1:9400"`

J2 — Single device (A1)

No. The only end-to-end demo, `examples/full-demo/run_demo.sh`, explicitly bypasses the shipped config. The shipped config refuses `auth_start` with `workos_not_configured`. A1 is officially NO in the project's own session log.

The composite task the product promises — capture, route, display on one machine — has no test and no documented procedure.

J3 — Two devices (A2)

No. No shipped two-device procedure exists. Federation library code carries `#[allow(dead_code)]`. Both "two node" test files start both daemons on `127.0.0.1`, so they prove the wire plane, not cross-device behaviour.

A user with two machines has no documented sequence to follow and no way to discover the pairing handshake.

J4 — GUI

Not from a clone. The GUI binary lives at `crates/fabric-gui/src-tauri/`, which root `Cargo.toml:5` **excludes** from the workspace. `cargo build --workspace` does not build it. The only documented delivery path is the `build-gui` GitHub Actions job producing OS installers.

Note the precision here: `fabric-gui` **is** a workspace member (`Cargo.toml:17`) and has **zero tests**. `src-tauri` is the excluded part. So the GUI is neither tested nor built by the standard command.

J5 — What is the product

The stated product (`README.md:9`):

> *"A packaged, all-in-one computing environment that makes connected physical machines, operating-system sessions, virtual machines, accelerators, displays, audio devices, input devices, storage, and remote locations appear as one coherent interactive computer."*

What the shipped tree actually is: a daemon, a CLI, a TUI, a tray app, a web frontend, and an **in-process** federation and graph library.

The gap between the two is twofold: 1. the two-device wire arm that was never shipped, and 2. an identity provider (WorkOS) that a fresh cloner cannot provision.

So the product claim requires two things a new user cannot supply.

The first thing that stops a new user

> *The documented start command fails on an empty secret, and the command that works is not documented anywhere.*

That is a one-session fix (`P2c` in the forward path) and it is the highest usability-per-effort item in the project.

Honest note on what was not tested

No GUI was launched in this audit. Nine HTML pages totalling 2,608 lines exist. Whether they render correct state against a live daemon is **UNKNOWN**, not “broken”. The distinction matters: this report does not claim the GUI is non-functional, only that nothing proves it is functional.

03 — Stub, Fake, and Silent No-Op Inventory

Commit: `c9eb1a3` · **Scope:** production code only (`crates/*/src/**`), 52,353 LOC across 209 files **Tests, examples, benches excluded** from both findings and the LOC denominator

Headline

Severity	Count	Meaning
BLOCKER	3	wired into a real entry point, reports success, does nothing
DEGRADED	9	works but limited, or a flag with no effect
COSMETIC	6	stale comment, dead `pub fn`, no behavioural impact
Total	**18**	**0.034% of production LOC**

Classic stubs: zero. No `todo!()`, no `unimplemented!()`, no `panic!("not implemented")`, no `.expect("TODO")` anywhere in production source. All 38 `panic!` occurrences in `crates/**/src/**` live inside `#[cfg(test)]` blocks or `#[test]` functions.

This matters: PhenoFabric does not have the usual "unfinished code loudly marked as unfinished" problem. It has the opposite and more dangerous problem — **finished-looking code that does nothing.**

BLOCKERs

B1 — `send_keys` accepts input and types nothing

`crates/fabric-terminal/src/web_main.rs:272–303`, registered as `POST /api/panes/{pane_id}/send` at line 688.

The function body carries its own admission:

```
// In production: execute tmux send-keys via subprocess
// For now: just log and broadcast to connected WebSockets
```

`for now` has shipped. An HTTP client POSTs keystrokes, receives **200 OK**, and the keystrokes are never typed into the pane. WebSocket subscribers receive an `input` event, so a UI that only listens to the socket may *appear* to work while the underlying terminal never changes.

B2 — `broadcast_local` promises remote pane injection, injects nothing

`crates/fabric-terminal/src/relay.rs:12–29`. The entire body is one `tracing::debug!` and `Ok(())`.

Its own docstring (`relay.rs:1–4`) promises: *"Takes remote pane content and injects it into the local tmux session. This enables viewing remote panes locally without SSH attachment."*

Comments at lines 22-26 enumerate the four steps a real implementation would take (find the local pane, clear it, write content line by line, set cursor). None execute. It is called from `crates/fabric-terminal/src/main.rs:173`, so any data path ending here silently succeeds.

B3 — `self-update` exits 0 having done nothing

`crates/fabric-capture/src/main.rs:45-49`.

The `#[cfg(feature = "self-update")]` gate is placed **inside** the match arm, gating only the `self_update::run(...)` call — not the trailing `return Ok(())`. And **no crate in the workspace ever enables the feature**. Result: `tf-win-capture self-update` matches the arm, skips the work, returns `Ok(())`, exits 0. The user is told nothing.

The same feature, declared for `fabric-terminal` at `Cargo.toml:19` and gated at `web_main.rs:27,540`, is likewise never enabled — that body is dead in every published build.

Why these three share a shape

All three are **stubs wired into real production entry points that report success.** They are invisible to `cargo check`, invisible to `cargo clippy`, and invisible to the test suite, because nothing asserts on the side effect that never happens.

This is the highest-value class of fix in the repository: each is small, each removes a false capability claim, and together they teach the codebase to fail loudly.

DEGRADED

Location	Finding
`fabric-terminal/Cargo.toml:19` + `web_main.rs:27,540`	`self-update` feature gates body code; never enabled; dead in every build
`fabric-orchestrator/Cargo.toml:20,30`	`surface-mojo` feature declared with an optional dep, but **no `#[cfg(feature = ...)]` exists anywhere in the crate** — the flag has literally no effect on compilation
`fabric-terminal/Cargo.toml:18`	`ssh` feature gates `dep:ssh2`; **is** enabled by default — this one works
`fabric-capture/src/main.rs:48,52`	Early-exit arms for `--version` and `self-update`
plus 5 further placeholder/deferred items	placeholder hardcoded values, deferred port checks, missing OS-family comparison, no WebSocket auth, dead `pub fn`s

Full line-level detail is in the lane output retained with this audit session.

Structural findings (not counted above, reported separately)

Finding	Location	Why it matters
Silent tier catch-all	`fabric-graph/src/multihop/mod.rs:211`	L3/L4/L8 all resolve to `"tcp_transport"` with no signal
No executor	workspace-wide `[#]`	`RouteExecutor` absent; selected transport never instantiated
`CopyPath` dead-but-wired	descriptor field	Carried, never matched; whatever it controlled is uncontrolled
Federation not on the wire	`fabric-daemon/src/federation/mod.rs:9-13`	`#[allow(dead_code)]` — the code admits unreachability

The honest summary

18 findings in 52,353 lines is a **low** density, and the classic-stub count is **zero**. The repository is not sloppy.

But the three BLOCKERs are all in user-facing surfaces (`fabric-terminal` HTTP API, `fabric-terminal` relay, `fabric-capture` CLI), all silent, and all report success. A user who encounters them concludes the feature works and the *system* is at fault — the worst possible failure mode, because it sends them debugging the wrong layer.

Fix order: B3 (smallest, isolated), B1, B2. All three are 1-2 sessions total.

04 — Quality Reality

Commit: `c9eb1a3` · **Observed in audit:** 751 unit, 63 integration, clippy clean, fmt clean, manifest 1086 entries verified

Question answered: does the passing test count prove the project is ready?

Short answer: no. The pass count is real. The *readiness claim* it is used to support is overstated, for three specific and fixable reasons.

---

1. Four crates have zero tests

Of 22 workspace members:

Crate	Unit	Integration	Total
`fabric-graph`	113	52	165
`fabric-frame-transport`	147	4	151
`fabric-daemon`	406	8	414
`fabric-capability`	21	29	50
`fabric-persist`	36	0	36
`fabric-cli`	22	13	35
`fabric-checker`	17	0	17
`fabric-graph-cli`	7	8	15
`phenotype-nvms-adapter`	13	0	13
`fabric-web`	5	0	5
`fabric-tray`	4	0	4
`fabric-gui`	**0**	**0**	**0**
`fabric-tui`	**0**	**0**	**0**
`phenotype-manifest`	**0**	**0**	**0**
`fabric-capability-ffi`	**0**	**0**	**0**

`fabric-gui` is the **desktop GUI** — 2,608 lines of HTML plus a Tauri shell — with zero tests. `fabric-capability-ffi` is a **C ABI** with zero tests. These are the two most externally-facing surfaces in the workspace and neither has a single assertion.

The workspace-wide pass count hides them completely.

---

2. The 751 figure is inflated

`cargo test --workspace --lib` is what CI runs (`ci.yml:211`). `--lib` only compiles library targets. Modules reachable only from `main.rs` are **silently skipped**.

Affected: `fabric-capture`, `fabric-tui`, and several `fabric-daemon` modules (`metrics`, `logging`, `input_guard`, `rate_limiter`, `secret_loader/*`).

Estimated **~83 unit tests never execute** under the CI command. The true unit test count is higher than 751, but the number CI reports is not the number that ran against the library.

This is not fraud; it is a flag mismatch. But it means “751 passing” is not the coverage statement it looks like.

---

3. Most “integration” tests are unit tests renamed

Of 27 `tests/*.rs` files, only **6 actually cross a real OS boundary** (real subprocess, real socket, real file). Roughly **73%** of the “integration” coverage is single-process public-API calls that never leave the test binary.

Both “two node” test files start two real daemons — but both bind `127.0.0.1`. They cross a process boundary, not a machine boundary. They are honest about what they do; they are not evidence of cross-device behaviour.

---

4. What is clean

Dimension	Finding
Mocks / fakes in production code	**None.** Verified workspace-wide
Classic stubs	**Zero** (`todo!`, `unimplemented!`)
Property-based testing	Real `proptest` usage, not decorative
E2E	Real TCP and real SQLite, not simulated
CI	3 workflows, mostly honest

---

5. CI soft spots

Workflow	Line	Issue
`deny.yml`	74, 78, 82	`govulncheck ... || true` on three lines — a vulnerable dependency cannot fail this check
`spec-validation.yml`	109	`go run . testdata/... || true`
`e2e.yml`	—	gating depends on GitHub branch protection, not on the workflow itself
`release.yml`	—	`continue-on-error: true` on macos-x86_64 and docker-build — documented and intentional

The `|| true` instances matter: they mean two checks *report* rather than *gate*. A report that cannot fail is not a control.

---

6. The five most important untested behaviours

Judged by user impact:

1. **Daemon signal handling is not wired.** Ctrl-C behaviour is not what the handler implies.
2. **Graceful shutdown does not drain connections.** In-flight frames can be lost on stop.

3. **Concurrent SQLite WAL writers** — no test crosses two writers.
4. **Wire protocol version mismatch** — no test asserts a clear refusal between versions.
5. **Capability revocation does not propagate to a live handle** — revocation is recorded, not enforced on an open session.

Every one of these is a data-loss or security-shaped gap, and every one is invisible to the current suite.

Verdict

Claim	Verdict
“Test suite passes”	**TRUE** — 751 unit, 63 integration, observed
“Code quality is high”	**TRUE** — clippy clean, no mocks in production, no classic stubs
“The suite proves readiness”	**FALSE** — four zero-test crates, ~83 skipped unit tests, 73% of “integration” is in-process
“Green CI means shippable”	**FALSE** — two checks are `|| true`, gating is external

The correct reading: this is a **well-engineered codebase with an under-powered verification net in exactly the places that face users.** The tests are honest; the summary metric is misleading.

05 — Documentation Trust

Commit: `c9eb1a3` · **Method:** claim extraction then verification against code **Sample:** 33 line-level claims across 25 rows, drawn from top-level docs, release notes, spec files, and the single runbook

Verdict counts

Verdict	Count	Share
WRONG (contradicts code incl. non-existent paths)	13	~39%
STALE (was true, drifted)	9	~27%
UNVERIFIABLE (not contradicted, not checked)	6	~18%
ACCURATE	5	~15%

WRONG + STALE = 55% of the sampled documentary evidence layer is factually wrong or out of date. Only 15% verified accurate.

At source-document level: **5 documents contain WRONG claims** (R0 release note, R1 release note, `specs/025-wire-transport/spec.md`, the 024-025 wire mapping, `DEVELOPMENT_GUIDE.md`).

The three most damaging inaccuracies

1. `docs/releases/2026-09-08-R1.md` — the R1 release evidence file

Most damaging because it is the document a stakeholder reads as the canonical “what was delivered” record. It describes delivered capability that the code does not contain. A reader who trusts it will believe the system is further along than it is.

2. `specs/025-wire-transport/spec.md` + `architecture/spec-024-025-wire-mapping.md`

These describe a wire surface whose message set and test counts do not match the code. A developer implementing against this spec will build against a fiction.

3. `config/settings.toml` (as pointed to by the WorkOS runbook)

The runbook’s commands lead a user to a config whose auth is enabled with an empty secret, so the documented path fails at the first step. The config is the thing that turns a documented procedure into a broken one.

Runbook commands that fail if followed today

The single runbook in the repo is `docs/runbooks/workos-infisical-shared.md`. Following it literally leads to `workos_not_configured`, because the shipped `config/settings.toml` has `[auth] enabled = true` and an empty `workos_client_secret`.

This is the worst category of documentation defect: **a user follows the instructions exactly and it does not work.**

Structural doc problems

Category	Finding
Paths cited as files that are directories	several
Aspirational layouts described as present	a proposed crate/file layout never implemented
Feature descriptions contradicting live behaviour	in-tree files describing modes the code does not have
Config keys silently ignored	at least one key read by nothing
Test counts that do not match	claimed counts differ from actual
LoC claims with small deltas	drift, low impact

The honest reading for the owner

The documentation is **not dishonest. It is a decision and specification layer being read as a status layer.**

The signals are all present if you look for them:

- `README.md:6` — “Status: Research and architecture baseline; not an implementation-complete claim”
- `STATUS.md:4` — “Lifecycle state: `specified`”

Those two lines are accurate. But release notes, spec files, and the runbook speak in the present tense about delivered capability. A reader who starts at the README is well served. A reader who starts at the release notes is misled.

Recommendation: do not write new capability docs until the corresponding code lands. Instead, add one authoritative `STATUS.md`-derived truth table (this report set can serve) and mark release notes as *planning records*, not delivered evidence. Retracting or annotating R0 and R1 is higher value than writing anything new.

Can the docs be trusted as a description of current state?

Document class	Trustworthy as current state?
`README.md` status lines	Yes
`STATUS.md` lifecycle state	Yes
`docs/adr/`	Yes as *decisions*, not as *implementations* — 35 ADRs, a large fraction partially or entirely unimplemented
Release notes R0 / R1	**No**
`specs/025` + wire mapping	**No**
`DEVELOPMENT_GUIDE.md`	**No**
`docs/runbooks/workos-infisical-shared.md`	**No** — commands fail as written

Bottom line: the ADRs and the README status lines are trustworthy. The release notes and spec files are not. This report set should be treated as the current status layer until code catches up to the documents.

06 — Forward Path

Commit: `c9eb1a3` · **Effort unit:** focused engineering sessions (one uninterrupted block, one concern) **No dates. No hour counts.** Effort is a range because the project forbids fabricated durations.

The ordering principle

> *Smallest remaining effort, fastest useful outcome, fewest dependencies.*

A small candidate is not the same as finishing a large repository. Each stage below is scoped so it can be accepted or rejected on its own evidence.

Stage overview

Stage	Work	Sessions	Depends on	Acceptance evidence
P0	Fix 3 silent-no-op BLOCKERs	1–2	none	each now fails loudly or works
P1	`RouteExecutor` seam + TCP fallback	1	none	plan executes; delta provably zero
P2a	`unix_socket` adapter + benchmark	2	P1	two daemons cross via UDS, measured
P2b	Federation wire arm	1–2	P1	`sync_topology` round-trips on the wire
P2c	First-run quickstart	1	none	clone → running in ≤3 commands
P3	Netns harness + replay	2	P2b	frames cross namespaces
P4	QUIC then SHM adapters	7–10	P2a	3 measurable transport deltas
P5	GUI headless smoke + composite test	2–3	none	page asserts live daemon state
P6	Human WorkOS sign-in	minutes (human)	P5	scoped user state observed

Path to a usable CVP: P0 + P1 + P2a + P2b + P2c = 7–11 sessions.

P0 — Fix the three silent no-ops

Why first: they are small, isolated, and each removes a false capability claim. They also establish the discipline of failing loudly, which the rest of the work depends on.

Item	File	Fix
B3 `self-update` exits 0	`fabric-capture/src/main.rs:45-49`	wrap the whole arm in the cfg, not just the call; return an explicit "feature not compiled in" error
B1 `send_keys` types nothing	`fabric-terminal/src/web_main.rs:272-303`	implement the tmux invocation, or return 501 so the caller knows
B2 `broadcast_local` injects nothing	`fabric-terminal/src/relay.rs:12-29`	implement the four documented steps, or return an error

Acceptance: a test asserts each path either performs its side effect or returns a non-success status. No path may return `Ok(())` having done nothing.

P1 — The keystone: `RouteExecutor`

This is the single most important piece of work in the project. Everything in A2 and A3 is built on it, and nothing currently exists.

```
crates/fabric-graph/src/multihop/executor.rs   (new)

pub trait RouteExecutor {
    async fn execute(&self, plan: &RoutePlan, buf: &mut FrameBuffer) -> Result<ExecOutcome>;
}
```

1. Define the trait. One method, takes a compiled plan and a buffer.
2. Implement `TcpExecutor` by delegating to the existing wire server. Behaviour must be **byte-identical** to today.
3. Call the executor from the daemon's session loop where `RoutePlan` is currently dropped.
4. Add a test proving `execute()` with a TCP plan produces the same bytes as the direct path.

Why a TCP fallback first: it proves the seam without changing behaviour, so the change cannot regress anything. It also makes the honest point that **today the transport delta is provably zero**, which is a real finding, not a gap to hide.

Acceptance: `RoutePlan` is no longer dead data; a test asserts the executor is invoked.

P2a — `unix_socket` adapter + first honest benchmark

1. `UnixSocketExecutor` using `std::os::unix::net` — no new dependency.
2. Test: two daemons on one Linux host, frames cross via UDS instead of TCP loopback.
3. Benchmark harness `tests/transport_compare.rs`: same frame stream at 1280×720, TCP vs UDS.

Acceptance: a measured delta exists. This is the first point at which "transport selection" is more than a string in a catalog.

Note: `unix_socket` is currently a *label* with no implementation, so this stage also fixes the `select_stages` L0/L1/L2/L5 mapping by giving it something real to point at.

P2b — Federation wire arm

The federation library (860 lines) carries `#[allow(dead_code)]` because nothing calls it.

1. Add `federation_sync_request` / `federation_sync_response` to `wire/protocol.rs`.
2. Route through `network_handlers`; invoke `sync_topology` in the session loop.
3. Drop the `#[allow(dead_code)]` on `federation/mod.rs:14` **only when** the arm exists.

Acceptance: a topology merge round-trips over the real wire.

Coordination: this touches `wire/`, which is the active development area. Sequence it after P1 so the executor exists first.

P2c — First-run quickstart

One session, highest usability-per-effort in the project.

1. A quickstart that uses the path that actually works (`cargo run -p fabric-daemon`, no `--config`) and says plainly that this is unauthenticated local mode.
2. A second section for the authenticated path that states the required env var and that it needs a WorkOS secret.
3. Fix `install.sh:6` or mark the installer unmaintained.
4. A `fabric-cli` tutorial for the 11 subcommands.

Acceptance: a person who has never seen the repo can start the daemon and run `fabric-cli status` in three commands.

P3 — Namespace harness

1. Two Linux netns joined by a veth pair; two daemons, one per namespace.
2. Userspace STUN stub returning the veth peer IP.
3. Replay the loopback two-node tests under the harness.
4. `#[cfg(target_os = "linux")]` gate so macOS/Windows CI skips cleanly.

Acceptance: A2 accepted without two physical machines.

P4 — QUIC then SHM

Step	Work	Sessions
QUIC adapter	`quinn` behind `quic-transport` feature; same trait shape	3–4
SHM adapter	`memfd_create` + `memmap2`, Linux only	4–6

Acceptance: three measurable transport deltas (TCP, UDS, QUIC), then a fourth (SHM).

P5 — GUI proof

`fabric-gui` has zero tests and 2,608 lines of HTML. Nothing proves it renders live state.

1. Add `headless_chrome` or Playwright as a dev-dependency.
2. Load `index.html`, navigate to `dashboard.html`, assert JS successfully calls the wire server.
3. One composite test: start daemon, drive a synthetic surface through `SurfaceRegistry`, open the GUI headlessly, assert the surface appears.

Acceptance: A1 code portion closed with evidence rather than assertion.

P6 — The one thing no agent can do

The WorkOS AuthKit sign-in requires a real browser, a real email inbox, and a redirect.

Step	Who
Open authorize URL	human
Submit email at `significant-vessel-93-staging.authkit.app`	human
Retrieve 6-digit code from inbox	human
Submit code, allow redirect to local callback	human
Everything after the callback	daemon

There is **no headless, device-code, or API-only path**. A1 acceptance is blocked on this and on nothing else.

What is deliberately NOT in this plan

Excluded	Why
Scaling work	operator scope exclusion
Sunset / deprecation	operator scope exclusion
User-feedback-driven change	operator scope exclusion, and the product is untested by a real user
Intent evolution	operator scope exclusion
PCIe / RDMA / IVSHMEM / vhost / DMA-BUF	hardware not present on this host — **UNKNOWN effort**
Rewriting anything already green	would regress a working baseline for no gain

---

The one decision the owner should make

`select_stages` returns `"tcp_transport"` for L3/L4/L8. Two defensible repairs:

Option	Behaviour	Cost	Consequence
A. Fail loudly	requesting PCIe/RDMA returns an explicit "unsupported" error	small	honest; may break callers that silently relied on fallback
B. Observable fallback	falls back to TCP but returns the actual transport used	small	honest *and* backward-compatible

Recommendation: B. It preserves behaviour, removes the silence, and gives the benchmark harness something to record. There is no repo evidence stating the intended behaviour, so this is a genuine product decision, not a technical one. Recorded as **UNKNOWN intent** rather than guessed.

07 — Method and Limits

Purpose: so the owner can calibrate how much to trust each number in this set.

How this audit was performed

Seven independent read-only lanes, each with a self-contained brief and a report obligation:

Lane	Task	Scope
`quokka`	stub / fake / laziness inventory	production code, 52,353 LOC
`ocelot`	usability journey tracing	5 end-to-end journeys
`marten`	test and CI honesty	22 crates, 5 workflows
`gecko`	goal status + ADR survey + effort	A1/A2/A3, 35 ADRs
`lynx`	documentation trust	33 claims across 25 rows
`tapir`	A1 live-run reconnaissance	WorkOS auth path
`marmot`	A3 transport selection forensics	route compiler

Every lane was read-only: no edits, no commits, no pushes. Lanes were forbidden from touching `crates/fabric-daemon/src/wire/`, which was under concurrent development.

Parent-side re-verification. Two of the most consequential claims were independently re-verified by the coordinating session rather than accepted from a lane:

Claim	Method	Result
`_ => "tcp_transport"` catch-all exists	direct `grep`	**confirmed** at `multihop/mod.rs:211`
`RouteExecutor` absent	workspace-wide search	**confirmed absent**
`fabric-gui` workspace membership	direct read of `Cargo.toml`	**is a member (line 17)**; only `src-tauri` excluded

That third check **corrected an earlier claim made during this ownership session** and is recorded in `00-EXECUTIVE-SUMMARY.md` §7.

What was NOT verified

This is the most important section in the set.

Not verified	Why	Consequence
Test suites were not run by lanes	deliberate, to avoid build contention with concurrent work in `wire/`	test counts come from the parent's own verified runs on `c9eb1a3`, not from lane execution
No GUI was launched	no human click available	the GUI is **UNKNOWN**, not "broken". 2,608 lines of HTML exist; whether they render live state is unproven
No `wire/protocol.rs` enumeration by the A2 lane	file was in the forbidden active area	the claim "no federation arm exists in protocol.rs" rests on the module's own `#[allow(dead_code)]` self-admission, not a direct read
No hardware transports tested	no RDMA NIC, no VFIO-passthrough GPU on this host	PCIe / RDMA / IVSHMEM effort is **UNKNOWN**, not estimated
28 of 35 ADRs characterized by summary, not deep-read	time-boxed	recommendation: deep-read ADRs 0030, 0031, 0033, 0034 before A2/A3 work
No performance measurement taken	nothing to measure — no executor exists	any future benchmark must run on the deployment hardware; transport deltas are platform-dependent

How to read the effort ranges

Effort is given in **focused engineering sessions**. One session is one uninterrupted block on one concern.

No wall-clock hours and no calendar dates are given, for two reasons:

1. The project forbids fabricated durations.
2. A calendar estimate would require a staffing assumption that is not the auditor's to make.

To convert: multiply sessions by the responsible engineer's real sessions-per-week, then add roughly 30% for review and iteration. Ranges assume one engineer with no other concurrent work on this repository.

Confidence by finding class

Finding class	Confidence	Basis
BLOCKER stubs (3)	**High**	each read at file:line, each has a quoted body
`RouteExecutor` absent	**High**	search-verified twice
Stub density (0.034%)	**High**	LOC denominator computed explicitly
Zero-test crates (4)	**High**	per-crate test count table
`--lib` skips ~83 tests	**Medium**	estimated from main-only module inventory, not a differential run
“73% of integration tests are in-process”	**Medium**	classification of 27 files, judgement applied per file
Docs 55% wrong/stale	**Medium**	33-claim sample; the sample is not the whole corpus
Effort ranges	**Low–Medium**	engineering judgement; hardware-gated parts explicitly UNKNOWN
A1 requires a human	**High**	no headless/device-code path found after directed search
GUI functional status	**UNKNOWN**	never launched

Known biases in this audit

1. **The lanes were told to be blunt.** That produces a report that reads more negative than the code deserves. The correct calibration is in `00-EXECUTIVE-SUMMARY.md` §3: the code is well engineered, the verification net is thin in user-facing places, and the wiring is unfinished.
2. **Read-only audits cannot prove absence perfectly.** “No executor exists” is a strong search result, not a proof. It is reported with high confidence because two independent searches agreed and the federation module self-admits the same gap.
3. **Effort estimates cluster around the auditor’s mental model of the codebase**, having read it. Inflate for unfamiliarity if a different engineer executes.

Reproducing this audit

```
# Stub inventory
rg -n 'todo!|unimplemented!|panic!\("not implemented"\)|expect\("TODO' crates/*/src/

# Per-crate test counts
for c in crates/*/; do printf '%-40s %s\n' "$c" \
  "$(rg -c '#\[(test|tokio::test|rstest)\]' "$c/src" 2>/dev/null | awk -F: '{s+=$2} END{print s+0}')";
        done

# Executor absence
rg -n 'execute_route|run_route|apply_route|run_plan|RouteExecutor' crates/*/src/

# The catch-all
rg -n 'tcp_transport' crates/fabric-graph/src/multihop/mod.rs

# Zero-test crates
for c in crates/*/; do n=$(rg -c '#\[test\]' "$c" 2>/dev/null | awk -F: '{s+=$2} END{print s+0}'); \
  [ "${n:-0}" -eq 0 ] && echo "ZERO TESTS: $c"; done
```

What would change this report's conclusions

If this were true	Then
A `RouteExecutor` exists under a name this search missed	A3 is far closer than stated; re-run the search first
The GUI renders correct live state when launched	A1's code gap is much smaller
L3/L4/L8 are documented elsewhere as intentional TCP fallback	the catch-all is a design decision, not a defect — but the silence remains one
Test lanes ran the suite and it disagreed with 751/63	the quality section needs revision
The deployment host has RDMA and a passthrough GPU	A3's upper bound stops being UNKNOWN